智能笔记助手 · 2 组 Git / GitHub 协作规范

- 版本：V2.0
- 编制：郭谦（2 组组长）
- 日期：2026-10-17
- 适用：2 组全体成员
- 仓库：**2 组自有仓库**（Public），地址见根目录 `README.md`

> V2.0 变更：lzt 已明确后续代码由两组**独立开发**，2 组改为**自建仓库**。因此 V1.0 中"借用 1 组仓库 / 需要 lzt 加权限"的内容全部作废，改为本版。

---

1. 为什么必须单独定这份规范

1 组 4 人同宿舍，可以线下商量、由一个人统一提交，不写规范也能跑。

2 组 5 人**分散在不同宿舍/不同时间**，没有线下协调条件，只能靠**显式规范**。规范缺一条，就多一次覆盖、冲突或返工。

---

2. 仓库归属与目录职责

2 组的仓库由**郭谦作为 owner**，组员可拿到写权限。仓库内部按来源分两块，边界必须清楚：

目录	归属	权限	说明
阶段成果/需求调研/**、阶段成果/需求建模/旧版废稿/**、阶段成果/前期实验/**	前期资料	建议只读	前期调研、共同基础件与实验记录；引用时标注来源，通常不直接在原件上改
阶段成果/需求建模/结果材料/**	2 组	读写	2 组本轮建模产出
src/**、工作区/需求分析/** 等	2 组	读写	后续阶段新增

规则：只改自己工作包对应的子目录；阶段成果/需求调研/、阶段成果/需求建模/旧版废稿/ **与** 阶段成果/前期实验/ **默认视为参考材料，不作为本轮主工作区。** 所有组员都在 阶段成果/需求建模/结果材料/ 下各有独立子目录（分工见任务分工书），因此冲突概率极低。

---

3. 仓库初始化（郭谦执行，一次性）

3.1 在 GitHub 建仓库

- 名称建议：`Intelligent-Reader-2zu`
- 可见性：**Public**
- **不要**勾选 "Add a README" / ".gitignore" / "license"（本地已有）

3.2 拉组员进仓库 Settings → Collaborators → Add people，把 4 名组员（程禹轩、朱紫轩、廖欣豪、汪昊）加成 **Write**。

3.3 开启主干保护 Settings → Branches → 对 `main` 添加规则，勾选 **Require a pull request before merging**。小团队若嫌繁琐，可退一步：不做强制保护，但**约定所有人都走 PR、不直推 main**。

3.4 本地切换到 2 组仓库 当前本地目录原本是 1 组仓库的克隆，需要切换为 2 组仓库。建议**重建历史**，让 2 组仓库的历史从零开始（前期材料已整理进 `阶段成果/`，不丢信息）：

```
# 把 1 组仓库的 git 记录改名备份（确认无误后可删）
Rename-Item .git .git-from-1组

# 初始化 2 组仓库
git init -b main
git remote add origin https://github.com/<你的用户名>/Intelligent-Reader-2zu.git
git remote add upstream https://github.com/Aut-172/Intelligent-Reader.git   # 保留 1 组仓库为上游，便于日后对照

# 首次提交
git add README.md .gitignore 阶段成果 docs
git commit -m "chore: 初始化 2 组仓库，归档前期共同材料"
git push -u origin main
```

> `upstream` 是可选但推荐的一步：日后若 1 组更新了调研材料，可用 `git fetch upstream` 对照，而不会污染 2 组的主干。

---

4. 分支策略

- `main`：**只接受 PR 合并**，不直接 push。
- 工作分支：**一人一分支一主题**，命名 `2zu/<姓名拼音>/<主题>`。

成员	示例分支
郭谦	`2zu/gq/model-v03`
廖欣豪	`2zu/lxh/state-diagram`
朱紫轩	`2zu/zzx/use-case-spec`
程禹轩	`2zu/cyx/ui-model`
汪昊	`2zu/wh/interview-evidence`

- 分支生命周期：从最新 `main` 切出 → 完成 → 提 PR → 合并后**立即删除**。
- 一个分支只做一件事，不要把"图形 + 文档"混在一个分支里。

---

5. 提交信息规范

格式：

```
<类型>(<范围>): <描述> [<任务编号>]
```

类型（取其一）

类型	含义
feat	新增内容（新图、新章节）
fix	修正基线中的错误
docs	文档类变更
chore	杂项（目录、配置、说明）
test	测试相关

范围：`2组/图形`、`2组/文档`、`2组/界面`、`2组/访谈`、`2组/模型`

任务编号：对应任务分工书的编号，如 `WP-A/A1`

示例

```
docs(2组/文档): 补全 UC01-UC12 用例规约 [WP-B/B1]
feat(2组/图形): 新增文档处理与回答生成状态图 [WP-A/A1-A2]
fix(2组/模型): 修正用例图 UC02 与 UC01 的关系方向 [WP1]
feat(2组/界面): 新增界面导航图与界面-用例映射 [WP-C/C3-C5]
chore(2组): 新增 2 组建模任务分工书与 Git 协作规范
```

硬要求

- 描述用中文，写清"做了什么"，不许出现"更新""修改""提交"这类无信息量的词；
- 一次提交只做一件事；
- `git add` 必须**按文件名加**，禁止 `git add -A` / `git add .`。

---

6. 标准工作流（每人每次照做）

```
# 1. 同步主干
git checkout main
git pull

# 2. 切自己的分支
git checkout -b 2zu/<姓名拼音>/<主题>

# 3. 干活，然后按文件名暂存
git add 阶段成果/需求建模/结果材料/06-建模图-2组-阅读助手/状态图-文档处理.puml

# 4. 提交（按第 5 节格式）
git commit -m "feat(2组/图形): 新增文档处理状态图 [WP-A/A1]"
```

```
# 5. 推送自己的分支
git push -u origin 2zu/<姓名拼音>/<主题>
```

6. 到 GitHub 上对该分支开 **Pull Request**，说明关联的任务编号；
7. 由**郭谦 Review** 通过后 **Squash merge** 进 main；
8. 合并后删除该分支。

---

7. 冲突处理

- 动手前先 `git pull --rebase origin main`，保持分支是最新的。
- 只改自己的子目录，冲突概率极低——**这也是目录隔离的意义**。
- 真冲突时**找郭谦处理，禁止自行 force push**。

---

8. 禁止事项（违反需回滚）

禁止	原因
直接 push 到 `main`	绕过评审，主干受保护
`git push --force` / `--force-with-lease`	会覆盖他人提交
修改 `阶段成果/需求调研/**`、`阶段成果/需求建模/旧版废稿/**`、`阶段成果/前期实验/**`	那些是前期资料，默认不作为本轮主工作区
提交访谈录音、姓名、学号、联系方式	仓库为 Public，合规风险
提交 `部分样本.zip`、`伪前端.zip`、`raw-local/`	含受限或个人信息
提交大于 10MB 的单个文件	仓库体积膨胀
`git add .` / `git add -A`	极易误带敏感文件

---

9. 组员接入方式（分三层，不要求所有人都会用 Git）

郭谦有可用 VPN，可直连 GitHub；组员情况不一，因此按能力分层：

通道	适用	做法
L1 直连	能正常访问 GitHub	按第 6 节走完整「分支 + PR」
L2 代提交	能读不能推，或没配好环境	产出打包发给郭谦，由郭谦推到**该成员自己的分支**，提交信息注明原作者
L3 交文件	完全零基础 / 无账号	只交成品文件（`.md` / `.puml` / `.png`），由郭谦统一入库

> 无论走哪一层，**分支名、提交信息、目录规范一视同仁**——代提交时沿用原作者的分支名，保证历史可追溯、分工可查证。

---

10. 备份与兜底

- **VPN 不稳定是本项目最大的工程风险**：它随时可能让 `push` / `pull` 失败。
- 应对：**先在本地 commit，联网后再 push**。本地仓库始终保有完整历史，网络中断不影响工作。
- 此外，`阶段成果/需求建模/结果材料/` 的全部产出建议在个人网盘/群文件**另存一份**，不依赖仓库单点。
- 每次合并到 main 后，郭谦在群里通报一次「已合并 <分支名> <提交号>」，保证全员知道主干状态。

---

11. 责任矩阵

事项	责任人
建仓库、拉组员、开启 main 保护	郭谦
下发编号规则 / 术语表、Review PR	郭谦
自建分支、规范提交、提 PR	每名成员
代提交（L2）、统一入库（L3）	郭谦
个人信息合规检查	郭谦（合并前逐次检查）
主干状态通报	郭谦